
Plantilla de Programación Competitiva

Universidad de Bogotá Jorge Tadeo Lozano

Facultad de Ciencias Naturales e Ingeniería

Semillero de Programación Competitiva

Maratón Nacional ACIS REDIS 2025

Elaborado por: Ludwig Alvarado.

Bogotá D.C., 17 de octubre de 2025

Índice

1. Leer Input	1
1.1. C++	1
1.1.1. n inicial y luego casos	1
1.1.2. Terminar cuando es 0 0	1
1.1.3. Leer líneas hasta EOF	1
1.1.4. Leer N líneas (string)	2
1.1.5. Leer vector	2
2. Output Format	2
2.1. C++	2
3. Macros	3
3.1. C++	3
4. Operaciones Bitwise	4
4.1. C++	4
5. Bitmask y Binarios	4
6. Swap dos números	6
7. Grafos: DFS, BFS y Dijkstra	6
7.1. Representación del Grafo	6
7.2. DFS (Depth First Search)	6
7.3. BFS (Breadth First Search)	7
7.4. Dijkstra (Shortest Path con pesos positivos)	8
7.5. Extras y Trucos de Grafos	8
8. Estructuras de Datos Built-In	10
8.1. Vector (<code>std::vector</code>)	10
8.2. Set y Multiset (<code>std::set</code> , <code>std::multiset</code>)	10
8.3. Map y Multimap (<code>std::map</code> , <code>std::multimap</code>)	11
8.4. Unordered Map / Set (<code>std::unordered_map</code> , <code>std::unordered_set</code>)	11
8.5. Deque (<code>std::deque</code>)	11
8.6. Stack / Queue / Priority Queue	12
8.7. Pair y Tuple	12
8.8. Algoritmos Comunes (<code><algorithm></code>)	12
8.9. Bitset (<code>std::bitset<N></code>)	13
8.10. String (<code>std::string</code>)	13
8.11. Union-Find / Disjoint Set (DSU)	13
8.12. Recomendaciones Finales	14

9. Expresiones Regulares (Regex) en C++	14
9.1. Incluir y estructuras básicas	14
9.2. Principales clases y funciones	14
9.3. Ejemplos comunes	15
9.4. Sintaxis básica de patrones	16
9.5. Ejemplos útiles en práctica competitiva	16
9.6. Trucos y advertencias	17
9.7. Ejemplo completo: contar palabras con formato específico	17
9.8. Resumen de usos típicos	17
10. Matemáticas y Teoría de Números	17
10.1. Operaciones Básicas	17
10.2. Aritmética Modular	18
10.3. Números Primos	18
10.4. Combinatoria y Factoriales Modulares	19
10.5. Algoritmos Numéricos Clásicos	20
10.6. Trucos y Consejos	21
10.7. Resumen de Complejidades	21

1. Leer Input

1.1. C++

1.1.1. n inicial y luego casos

```
1
2 int n; cin >> n;
3 while(n--){
4     int a, b; cin >> a >> b;
5     cout << (a + b) << endl;
6 }
```

```
3
1 2
5 7
6 3
—— out
3
12
9
```

1.1.2. Terminar cuando es 0 0

```
1 int a, b;
2 while(cin >> a >> b, (a || b))
3     cout << (a + b) << endl;
```

```
1 2
5 7
6 3
0 0
1 1 <— no se procesa
—— out
3
12
9
```

1.1.3. Leer líneas hasta EOF

```
1 string line;
2 while(getline(cin, line)){
3     stringstream ss(line);
4     int x, sum = 0;
5     while(ss >> x) sum += x;
```

```

6     cout << sum << endl;
7 }

```

```

3 4 5
10 20
7 8 9 10
—— out
12
30
34

```

1.1.4. Leer N líneas (string)

```

1 int N;
2 if (!(cin >> N)) return 0;
3 cin.ignore();
4 for(int i = 0; i < N; ++i){
5     string s;
6     getline(cin, s);
7 }

```

1.1.5. Leer vector

```

1 int n; cin >> n;
2
3 vector<int> v(n);
4 copy_n(istream_iterator<int>(cin), n, v.begin());

```

2. Output Format

2.1. C++

```

1
2 vector<int> v = {1, 2, 3, 4, 5};
3 copy(v.begin(), v.end(), ostream_iterator<int>(cout));
4 // 12345%
5
6 double x = 123.456789;
7 cout << fixed << setprecision(3);
8 cout << "Value: " << x << endl;
9 // Value: 123.457
10
11 cout << setw(10) << setfill(',') << 42 << endl;

```

```

12 // *****42
13
14 cout << boolalpha << (x > 100) << endl;
15 cout << hex << 255 << endl;
16 // true
17 // ff
18
19 vector<vector<int>> mat = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
20 for(auto &row : mat){
21     for (int i = 0; i < row.size(); ++i)
22         cout << (i ? " " : "") << row[i];
23     cout << endl;
24 }
25 /*
26 1 2 3
27 4 5 6
28 7 8 9
29 */

```

3. Macros

3.1. C++

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define fast ios::sync_with_stdio(false); cin.tie(nullptr);
4     cout.tie(nullptr)
5 #define endl '\n'
6 #define pb push_back
7 #define all(x) (x).begin(), (x).end()
8 #define sz(x) (int)(x).size()
9 #define for0(i, n) for (int i = 0; i < (int)(n); ++i) // base
10     0 index
11 #define for1(i, n) for (int i = 0; i < (int)(n); ++i) // base
12     1 index
13 #define forc(i, l, r) for (int i = (int)(l); i <= (int)(r);
14     ++i) // intervalo cerrado de l a r inclusivo
15 #define forr0(i, n) for (int i = (int)(n) - 1; i >= 0 ; --i)
16     // al contrario hasta 0.
17 #define forr1(i, n) for (int i = (int)(n) - 1; i > 0; --i) //
18     al contrario hasta 1
19
20 typedef long long ll;
21 typedef short int si;

```

```

16 typedef unsigned long long ull;
17 typedef long double ld;
18 typedef unsigned int ui;
19 typedef string str;
20 typedef pair<int, int> pii;
21 typedef pair<ll, ll> pll;
22 typedef vector<int> vi;
23
24
25 const int INFI = INT_MAX;
26 const long INFL = LONG_MAX;
27 const long long INFL = LLONG_MAX;

```

4. Operaciones Bitwise

4.1. C++

```

1
2 // Macros para operaciones bitwise
3 #define isOn(S, j) (S & (1 << j))
4 #define setBit(S, j) (S |= (1 << j))
5 #define clearBit(S, j) (S &= ~(1 << j))
6 #define toggleBit(S, j) (S ^= (1 << j))
7 #define lowBit(S) (S & (-S))
8 #define setAll(S, n) (S = (1 << n)-1)
9
10 #define modulo(S, N) ((S) & (N-1)) // Retorna S % N, donde N
    es una potencia de 2
11 #define isPowerOfTwo(S) (!(S) & (S-1))
12 #define nearestPowerOfTwo(S) (1 << lround(log2(S)))
13 #define turnOffLastBit(S) ((S) & (S-1))
14 #define turnOnLastZero(S) ((S) | (S+1))
15 #define turnOffLastConsecutiveBits(S) ((S) & (S+1))
16 #define turnOnLastConsecutiveZeroes(S) ((S) | (S-1))

```

5. Bitmask y Binarios

```

1 ll binpow(ll a, ll b){
2     ll res = 1;
3     ll counter = 0;
4     while (b > 0){
5         bitset<16> binary(b);
6         if ( b& 1 ) res = res * a;

```

```

7      a + a * a;
8      counter++;
9      b >>=1;
10     }
11     return res;
12 }
13
14 ll binpowmod(ll a, ll b, ll m){
15     a %= m;
16     ll res = 1;
17     while ( b > 0 ){
18         if ( b & 1 ) res = res * a % m;
19         a = a * a % m;
20         b >>= 1;
21     }
22     return res;
23 }
24
25
26 bitset<8> b1; // 00000000
27 bitset<8> b2(5); // 00000101
28 bitset<8> b3(string("1101")); // 00001101
29 bitset<8> b("10110010")
30 b[0] // 0 (least significant bit)
31 b[7] // 1 (most significant bit)
32 b[0] = 1; // Set bit 0 10110011
33 b.set(3); // set bit at index 3 10111011
34 b.reset(1); // reset bit at index 1
35 b.flip(2); // toggle bit at index 2 10111101
36 b.flip();
37
38 b.any() // True if any bit == 1
39 b.none() // true if all bits == 0
40 b.all() // true if all bits == 1
41 b.count() // number of bits == 1
42 b.size() // number of bits in bitset
43
44 b.to_ulong() // unsigned long
45 b.to_ullong() // unsigned long long
46 b.to_string() // string
47
48 // Obtener los subconjuntos de una mascara en binario
49 vector<int> subset_bitmask(int mask){
50     vector<int> subsets;

```

```

51     for (int subset = mask; subset; subset = (mask & (subset
52         -1))) subsets.push_back(subset);
53     return subsets;
54 }

```

6. Swap dos números

```

1 void swap_two_iters(vector<int> vec, int n){
2     const auto itr = find(all(vec), n);
3     auto indx = distance(vec.begin(), itr);
4     iter_swap(vec.begin() + n, itr);
5     iter_swap(vec.begin() + n, vec.begin() + indx);
6 }
7
8 int num = 2;
9 const auto itr = find(all(v), num);
10 auto indx = distance(v.begin(), itr);
11 iter_swap(v.begin() + num, v.begin() + indx);

```

7. Grafos: DFS, BFS y Dijkstra

Esta sección contiene implementaciones base y trucos para grafos no dirigidos o dirigidos, con pesos o sin ellos.

7.1. Representación del Grafo

```

1 // Grafo con N nodos numerados 0..N-1
2 int N;
3 vector<vector<int>>> adj(N); // sin peso
4 vector<vector<pair<int,int>>>> g(N); // con pesos (v, w)
5
6 for(int i=0;i<M;i++){
7     int u,v; cin>>u>>v;
8     adj[u].push_back(v);
9     adj[v].push_back(u); // quitar si es dirigido
10 }

```

7.2. DFS (Depth First Search)

Complejidad: $O(N + M)$

```

1 vector<vector<int>>> adj;
2 vector<int> vis;

```

```

3
4 void dfs(int u){
5     vis[u] = 1;
6     for(int v : adj[u])
7         if(!vis[v])
8             dfs(v);
9 }
10
11 int main(){
12     int n,m; cin>>n>>m;
13     adj.assign(n,{});
14     vis.assign(n,0);
15     while(m--){
16         int u,v; cin>>u>>v;
17         adj[u].push_back(v);
18         adj[v].push_back(u);
19     }
20     dfs(0); // desde nodo 0
21 }

```

Trucos:

- Útil para contar componentes conexas.
- Para detectar ciclos: pasar padre y verificar si hay arista a nodo visitado distinto del padre.
- Usar iterativo con stack si hay riesgo de stack overflow.

7.3. BFS (Breadth First Search)

Complejidad: $O(N + M)$

```

1 vector<vector<int>> adj;
2 vector<int> dist;
3
4 void bfs(int s){
5     queue<int> q;
6     dist.assign(adj.size(), -1);
7     dist[s] = 0;
8     q.push(s);
9
10    while(!q.empty()){
11        int u = q.front(); q.pop();
12        for(int v : adj[u])
13            if(dist[v] == -1){
14                dist[v] = dist[u] + 1;
15                q.push(v);

```

```

16         }
17     }
18 }

```

Trucos:

- Útil para hallar distancia mínima en grafos no ponderados.
- Para reconstruir camino: mantener vector `parent[v]`.

7.4. Dijkstra (Shortest Path con pesos positivos)

Complejidad: $O((N + M) \log N)$ con `priority_queue`.

```

1 vector<vector<pair<int,int>>> g; // (v, w)
2 const ll INF = 1e18;
3
4 vector<ll> dijkstra(int s){
5     vector<ll> dist(g.size(), INF);
6     priority_queue<pair<ll,int>, vector<pair<ll,int>>,
7         greater<pair<ll,int>>> pq;
8     dist[s] = 0;
9     pq.push({0,s});
10
11     while(!pq.empty()){
12         auto [du, u] = pq.top(); pq.pop();
13         if(du != dist[u]) continue;
14         for(auto [v, w] : g[u]){
15             if(dist[v] > du + w){
16                 dist[v] = du + w;
17                 pq.push({dist[v], v});
18             }
19         }
20     }
21     return dist;
22 }

```

Trucos:

- Si hay muchas aristas ($M \approx N^2$), usar vector de mínimos ($O(N^2)$).
- Usar `greater<pair<ll,int>> pq;` para min-heap.
- Para camino mínimo: mantener vector `parent[v]`.

7.5. Extras y Trucos de Grafos

1. Componentes Conexas:

```

1 int comp = 0;
2 for(int i=0;i<n;i++)
3     if(!vis[i]){ dfs(i); comp++; }

```

2. BFS 0-1 (para grafos con pesos 0 y 1):

```

1 deque<int> dq;
2 vector<int> dist(n, 1e9);
3 dist[s] = 0;
4 dq.push_back(s);
5
6 while(!dq.empty()){
7     int u = dq.front(); dq.pop_front();
8     for(auto [v,w]: g[u]){
9         if(dist[v] > dist[u] + w){
10             dist[v] = dist[u] + w;
11             if(w==1) dq.push_back(v);
12             else dq.push_front(v);
13         }
14     }
15 }

```

3. Detectar Ciclos en Dirigido (Toposort Check):

```

1 vector<int> indeg(n);
2 for(int u=0;u<n;u++)
3     for(int v:adj[u]) indeg[v]++;
4 queue<int> q;
5 for(int i=0;i<n;i++) if(!indeg[i]) q.push(i);
6 int cnt=0;
7 while(!q.empty()){
8     int u=q.front(); q.pop(); cnt++;
9     for(int v:adj[u]) if(--indeg[v]==0) q.push(v);
10 }
11 bool has_cycle = (cnt < n);

```

4. BFS multi-source:

```

1 queue<int> q;
2 for(int s: fuentes){
3     dist[s]=0;
4     q.push(s);
5 }

```

5. Bidirectional BFS (más rápido si inicio y fin conocidos).

6. Usar vector<vector<pair<int,int>>> si los pesos son necesarios.

7. Reservar memoria antes: adj.assign(n, {});

Resumen de Complejidades:

Algoritmo	Complejidad	Uso principal
DFS	$O(N + M)$	Conectividad, topología, ciclos
BFS	$O(N + M)$	Distancias en grafos no ponderados
Dijkstra	$O((N + M) \log N)$	Caminos mínimos (pesos positivos)
0-1 BFS	$O(N + M)$	Caminos mínimos (pesos 0/1)
TopoSort (Kahn)	$O(N + M)$	DAG orden topológico

8. Estructuras de Datos Built-In

Esta sección contiene las estructuras y algoritmos más útiles de la STL para programación competitiva (C++11). Incluye las operaciones más comunes y su complejidad temporal.

8.1. Vector (`std::vector`)

Array dinámico.

- Acceso por índice: $O(1)$
- Inserción/ Eliminación al final: $O(1)$ amortizado
- Inserción/ Eliminación en medio: $O(n)$
- Redimensionar: $O(n)$

```
1 vector<int> v;  
2 v.push_back(10);  
3 v.pop_back();  
4 v.resize(5);  
5 v.clear();  
6 v.reserve(1000); // evita realocaciones  
7 int x = v.back();
```

8.2. Set y Multiset (`std::set`, `std::multiset`)

Árbol balanceado (Red-Black Tree).

- Insertar/ Eliminar/ Buscar: $O(\log n)$
- `lower_bound` / `upper_bound`: $O(\log n)$

```
1 set<int> s;  
2 s.insert(3);  
3 s.insert(1);  
4 s.insert(2);  
5 s.erase(1);  
6 if(s.find(3) != s.end()) cout << "found\n";  
7  
8 for(int x : s) cout << x << " "; // 2 3
```

multiset permite duplicados y mantiene orden.

```
1 multiset<int> ms = {1, 1, 2, 3};
2 ms.erase(ms.find(1)); // borra solo una ocurrencia
```

8.3. Map y Multimap (std::map, std::multimap)

Diccionario ordenado por clave.

- Insertar/Eliminar/Buscar: $O(\log n)$
- Acceso por clave: $O(\log n)$

```
1 map<string, int> freq;
2 freq["apple"]++;
3 freq["banana"] = 3;
4
5 for (auto &p : freq)
6     cout << p.first << ": " << p.second << endl;
```

8.4. Unordered Map / Set (std::unordered_map, std::unordered_set)

Tablas hash (no ordenadas).

- Inserción/Eliminación/Búsqueda promedio: $O(1)$
- Peor caso: $O(n)$ (colisiones)

```
1 unordered_map<string, int> mp;
2 mp["dog"] = 5;
3 if(mp.count("dog")) cout << mp["dog"];
```

Recomendado para conteos rápidos o lookups cuando el orden no importa.

8.5. Deque (std::deque)

Cola doble: acceso eficiente en ambos extremos.

- push_front / push_back: $O(1)$
- pop_front / pop_back: $O(1)$
- Acceso por índice: $O(1)$

```
1 deque<int> dq;
2 dq.push_back(1);
3 dq.push_front(2);
4 cout << dq.front() << " " << dq.back(); // 2 1
```

8.6. Stack / Queue / Priority Queue

Stack: LIFO ($O(1)$ push/pop/top)

```
1 stack<int> st;
2 st.push(5);
3 cout << st.top(); // 5
4 st.pop();
```

Queue: FIFO ($O(1)$ push/pop/front)

```
1 queue<int> q;
2 q.push(1);
3 q.push(2);
4 q.pop();
5 cout << q.front(); // 2
```

Priority Queue: heap binario ($O(\log n)$)

```
1 priority_queue<int> pq; // max-heap
2 pq.push(10);
3 pq.push(5);
4 cout << pq.top(); // 10
5 pq.pop();
6
7 priority_queue<int, vector<int>, greater<int>> minpq; //
   min-heap
```

8.7. Pair y Tuple

Emparejar valores o agrupar varios tipos.

```
1 pair<int, string> p = {1, "abc"};
2 auto [x, y] = p;
3
4 tuple<int, int, string> t = {1, 2, "hi"};
5 auto [a, b, c] = t;
```

8.8. Algoritmos Comunes (<algorithm>)

- `sort(all(v))` $\rightarrow O(n \log n)$
- `reverse(all(v))` $\rightarrow O(n)$
- `binary_search(all(v), x)` $\rightarrow O(\log n)$
- `lower_bound(all(v), x)` $\rightarrow O(\log n)$
- `max_element(all(v))` $\rightarrow O(n)$
- `accumulate(all(v), 0)` $\rightarrow O(n)$

```

1 vector<int> v = {3, 1, 4, 2};
2 sort(all(v));
3 reverse(all(v));
4 if(binary_search(all(v), 3)) cout << "found";

```

8.9. Bitset (std::bitset<N>)

Arreglo de bits estático.

- Acceso/Set/Reset: $O(1)$
- Operaciones bitwise: $O(N/\text{word_size})$

```

1 bitset<8> b("10100110");
2 b.set(3);
3 b.reset(1);
4 b.flip();
5 cout << b.count(); // num de bits encendidos

```

8.10. String (std::string)

- Acceso: $O(1)$
- Comparar: $O(\min(n, m))$
- Substring: $O(k)$

```

1 string s = "hello world";
2 cout << s.substr(0,5);
3 reverse(all(s));
4 if(s.find("he") != string::npos) cout << "found";

```

8.11. Union-Find / Disjoint Set (DSU)

Estructura esencial para componentes conectadas o MST. Complejidad amortizada $O(\alpha(n)) \approx O(1)$.

```

1 struct DSU {
2     vector<int> p, sz;
3     DSU(int n): p(n), sz(n,1){ iota(p.begin(), p.end(), 0); }
4     int find(int x){ return p[x]==x ? x : p[x]=find(p[x]); }
5     bool unite(int a, int b){
6         a=find(a); b=find(b);
7         if(a==b) return false;
8         if(sz[a]<sz[b]) swap(a,b);
9         p[b]=a; sz[a]+=sz[b];

```

```

10         return true;
11     }
12 };

```

8.12. Recomendaciones Finales

```

1  // Siempre incluir
2  ios::sync_with_stdio(false);
3  cin.tie(nullptr);
4
5  // Alias y macros utiles
6  using ll = long long;
7  using pii = pair<int,int>;
8  #define all(x) (x).begin(), (x).end()
9  #define rall(x) (x).rbegin(), (x).rend()
10 #define sz(x) int((x).size())
11
12 // RNG
13 mt19937
    rng(chrono::steady_clock::now().time_since_epoch().count());

```

9. Expresiones Regulares (Regex) en C++

La biblioteca `<regex>` permite realizar coincidencias de patrones en cadenas usando expresiones regulares, útiles para validación, búsqueda y extracción de información.

9.1. Incluir y estructuras básicas

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      string s = "abc123xyz";
6      regex re("[0-9]+"); // uno o mas digitos
7      if (regex_search(s, re))
8          cout << "Tiene numeros!\n";
9  }

```

9.2. Principales clases y funciones

- `regex re("pattern");` — Compila un patrón.
- `regex_match(str, re)` — Comprueba si *toda* la cadena coincide.

- `regex_search(str, re)` — Comprueba si alguna *subcadena* coincide.
- `regex_replace(str, re, repl)` — Reemplaza coincidencias por otra cadena.
- `sregex_iterator` — Permite recorrer todas las coincidencias.

9.3. Ejemplos comunes

1. Coincidencia completa

```
1 regex re("[a-z]+");
2 cout << boolalpha << regex_match("abc", re) << endl; // true
3 cout << regex_match("abc123", re) << endl;           // false
```

2. Buscar dentro de una cadena

```
1 string s = "code123forces";
2 regex re("[0-9]+");
3 smatch m;
4 if (regex_search(s, m, re))
5     cout << "Primer numero: " << m.str() << "\n"; // "123"
```

3. Extraer todas las coincidencias

```
1 string text = "ID12 AB34 XY99";
2 regex re("[A-Z]{2}[0-9]{2}");
3 for (sregex_iterator it(text.begin(), text.end(), re), end_it;
4      it != end_it; ++it)
5     cout << it->str() << " "; // ID12 AB34 XY99
```

4. Reemplazar coincidencias

```
1 string s = "2025-10-17";
2 regex re("-");
3 string out = regex_replace(s, re, "/");
4 cout << out << "\n"; // 2025/10/17
```

9.4. Sintaxis básica de patrones

Patrón	Significado
.	Cualquier carácter
[abc]	Uno de a, b, c
[^ abc]	Cualquier excepto a, b, c
[a–z]	Rango de letras minúsculas
\d	Dígito (0–9)
\w	Alfanumérico o _
\s	Espacio, tab, salto de línea
^	Inicio de cadena
\$	Fin de cadena
a?	0 o 1 ocurrencia de a
a*	0 o más ocurrencias
a+	1 o más ocurrencias
a{3}	Exactamente 3 veces
a{2,4}	Entre 2 y 4 veces
(abc)	Grupo
ab—	a o b

9.5. Ejemplos útiles en práctica competitiva

Validar número entero o decimal

```
1 regex re("^-?[0-9]+(\\.[0-9]+)?$");
2 cout << regex_match("-12.34", re); // true
```

Detectar emails simples

```
1 regex email("[\\w.-]+@[\\w.-]+\\.[a-zA-Z]{2,}");
2 cout << regex_match("name@example.com", email); // true
```

Separar palabras por espacios

```
1 string s = "this is a test";
2 regex re("\\s+");
3 sregex_token_iterator it(s.begin(), s.end(), re, -1);
4 for (; it != sregex_token_iterator(); ++it)
5     cout << *it << "\n";
```

Filtrar líneas que contengan un patrón

```
1 regex pattern("ERROR");
2 string line;
3 while(getline(cin, line))
4     if(regex_search(line, pattern))
5         cout << line << "\n";
```

9.6. Trucos y advertencias

- Para **competencias**, preferir `string::find` o `substr` si el patrón es simple: `regex` puede ser lento.
- Usar `smatch` (para `string`) o `cmatch` (para C-strings).
- Los backslashes deben duplicarse: `"\\d+"` (no `"\d+"`).
- `regex_constants::icase` hace coincidencias sin distinguir mayúsculas.

9.7. Ejemplo completo: contar palabras con formato específico

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 int main(){
4     string text = "A1 B22 C333 D44";
5     regex re("[A-Z][0-9]{2,}");
6     int cnt = 0;
7     for(sregex_iterator it(text.begin(), text.end(), re),
8         end_it; it!=end_it; ++it)
9         cnt++;
10    cout << "Coincidencias: " << cnt << "\n"; // 3
11 }
```

9.8. Resumen de usos típicos

Tarea	Regex típico
Número entero	<code>^-?[0-9]+\$</code>
Decimal	<code>^-?[0-9]*\\.?[0-9]+\$</code>
Email simple	<code>[\\w.-]+@[\\w.-]+\\.?[a-zA-Z]{2,}</code>
Solo letras	<code>^[A-Za-z]+\$</code>
Tokenizar por espacios	<code>\\s+</code>
Buscar palabra específica	<code>\\bword\\b</code>

10. Matemáticas y Teoría de Números

Conceptos y algoritmos esenciales para resolver problemas de aritmética, combinatoria y teoría de números en programación competitiva.

10.1. Operaciones Básicas

```
1 ll gcd(ll a, ll b){ return b ? gcd(b, a % b) : a; }
2 ll lcm(ll a, ll b){ return a / gcd(a,b) * b; }
3 ll mod(ll a, ll m){ return (a % m + m) % m; } // siempre
   positivo
```

Trucos:

- Usar `__gcd(a,b)` de `<algorithm>`.
- Para overflow en multiplicación: usar `(__int128)` o reducción modular.

10.2. Aritmética Modular

Suma, Resta, Multiplicación

```
1 const ll MOD = 1e9+7;
2 ll add(ll a, ll b){ return (a + b) % MOD; }
3 ll sub(ll a, ll b){ return (a - b + MOD) % MOD; }
4 ll mul(ll a, ll b){ return (a * b) % MOD; }
```

Exponenciación Modular

```
1 ll modpow(ll a, ll e, ll m){
2     ll res = 1;
3     while(e){
4         if(e & 1) res = res * a % m;
5         a = a * a % m;
6         e >>= 1;
7     }
8     return res;
9 }
```

Inverso Modular (cuando m es primo)

```
1 // Fermat:  $a^{(m-2)} \bmod m$ 
2 ll modinv(ll a, ll m){ return modpow(a, m-2, m); }
```

Inversos del 1..n en $O(n)$

```
1 vector<ll> inv(n+1);
2 inv[1] = 1;
3 for(int i=2; i<=n; i++)
4     inv[i] = MOD - (MOD/i) * inv[MOD % i] % MOD;
```

10.3. Números Primos

Prueba de primalidad simple

```
1 bool is_prime(ll n){
2     if(n < 2) return false;
3     for(ll i=2; i*i<=n; i++)
4         if(n % i == 0) return false;
5     return true;
6 }
```

Criba de Eratóstenes

```
1 int N = 1e6;
2 vector<int> primes;
3 vector<bool> isprime(N+1, true);
4
5 void sieve(){
6     isprime[0] = isprime[1] = false;
7     for(int i=2; i*i<=N; i++)
8         if(isprime[i])
9             for(int j=i*i; j<=N; j+=i)
10                 isprime[j] = false;
11     for(int i=2; i<=N; i++)
12         if(isprime[i]) primes.push_back(i);
13 }
```

Factorización con Criba

```
1 vector<int> spf(N+1); // smallest prime factor
2
3 void sieve_spf(){
4     iota(spf.begin(), spf.end(), 0);
5     for(int i=2; i*i<=N; i++)
6         if(spf[i] == i)
7             for(int j=i*i; j<=N; j+=i)
8                 if(spf[j] == j) spf[j] = i;
9 }
10
11 vector<int> factorize(int n){
12     vector<int> f;
13     while(n > 1){
14         f.push_back(spf[n]);
15         n /= spf[n];
16     }
17     return f;
18 }
```

10.4. Combinatoria y Factoriales Modulares

Precomputar factoriales

```
1 const int MAXN = 1e6;
2 vector<ll> fact(MAXN+1), invfact(MAXN+1);
3
4 void init_fact(){
5     fact[0] = 1;
6     for(int i=1; i<=MAXN; i++) fact[i] = fact[i-1]*i % MOD;
7     invfact[MAXN] = modinv(fact[MAXN], MOD);
8     for(int i=MAXN; i>=1; i--) invfact[i-1] = invfact[i]*i % MOD;
```

```
9 }
```

Combinaciones nCr mod m

```
1 ll nCr(int n, int r){
2     if(r<0 || r>n) return 0;
3     return fact[n] * invfact[r] % MOD * invfact[n-r] % MOD;
4 }
```

Truco: Para $n \leq 10^6$, precomputar factoriales; para n grande pero r pequeño, usar fórmula multiplicativa.

10.5. Algoritmos Numéricos Clásicos

MCD extendido

```
1 ll extgcd(ll a, ll b, ll &x, ll &y){
2     if(b == 0){ x = 1; y = 0; return a; }
3     ll x1, y1;
4     ll g = extgcd(b, a % b, x1, y1);
5     x = y1;
6     y = x1 - y1 * (a / b);
7     return g;
8 }
```

Solución de Congruencia Lineal $ax \equiv b \pmod{m}$

```
1 ll solve_congruence(ll a, ll b, ll m){
2     ll x, y;
3     ll g = extgcd(a, m, x, y);
4     if(b % g) return -1; // no solution
5     x = (x * (b/g)) % m;
6     return (x + m) % m;
7 }
```

Criba para funciones aritméticas (ϕ)

```
1 vector<int> phi(N+1);
2 void sieve_phi(){
3     for(int i=0; i<=N; i++) phi[i]=i;
4     for(int i=2; i<=N; i++)
5         if(phi[i]==i)
6             for(int j=i; j<=N; j+=i)
7                 phi[j] -= phi[j]/i;
8 }
```

Conteo de divisores y suma de divisores

```
1 int num_div(int n){
2     int cnt=1;
3     for(int i=2, c; i*i<=n; i++){
```

```

4      c=0;
5      while(n%i==0) n/=i, c++;
6      cnt *= (c+1);
7  }
8  if(n>1) cnt*=2;
9  return cnt;
10 }
```

10.6. Trucos y Consejos

- **MODs comunes:** $10^9 + 7$, 998244353.
- Siempre usar $(a * 1LL * b) \% MOD$ para evitar overflow.
- Para **coprimos grandes**, el inverso modular existe.
- Para **conteo combinatorio**, usar logaritmos o Lucas si n es muy grande.
- **Euler's Totient:** $\phi(n)$ = cantidad de números $\leq n$ coprimos con n .
- **Exponenciación rápida** = base para muchos algoritmos (potencias, inversos, etc.).

10.7. Resumen de Complejidades

Algoritmo	Complejidad	Descripción
Criba Eratóstenes	$O(n \log \log n)$	Todos los primos hasta n
Factorización (con SPF)	$O(\log n)$	Usando preprocesamiento
Exp. Modular	$O(\log e)$	Potencia rápida
Inversos 1..n	$O(n)$	Todos los inversos mod primo
nCr con factoriales	$O(1)$	Tras precomputar factoriales